

```
import matplotlib.pyplot as plt
import numpy as np
from skimage.io import imread
from skimage.transform import downscale_local_mean
from skimage import img_as_uint
from skimage.color import rgb2gray
from skimage.color import gray2rgb
from skimage.color import rgb2hsv
from skimage.color import hsv2rgb
import cv2
from skimage.transform import resize
from skimage.util import img_as_ubyte
```

Analog Image

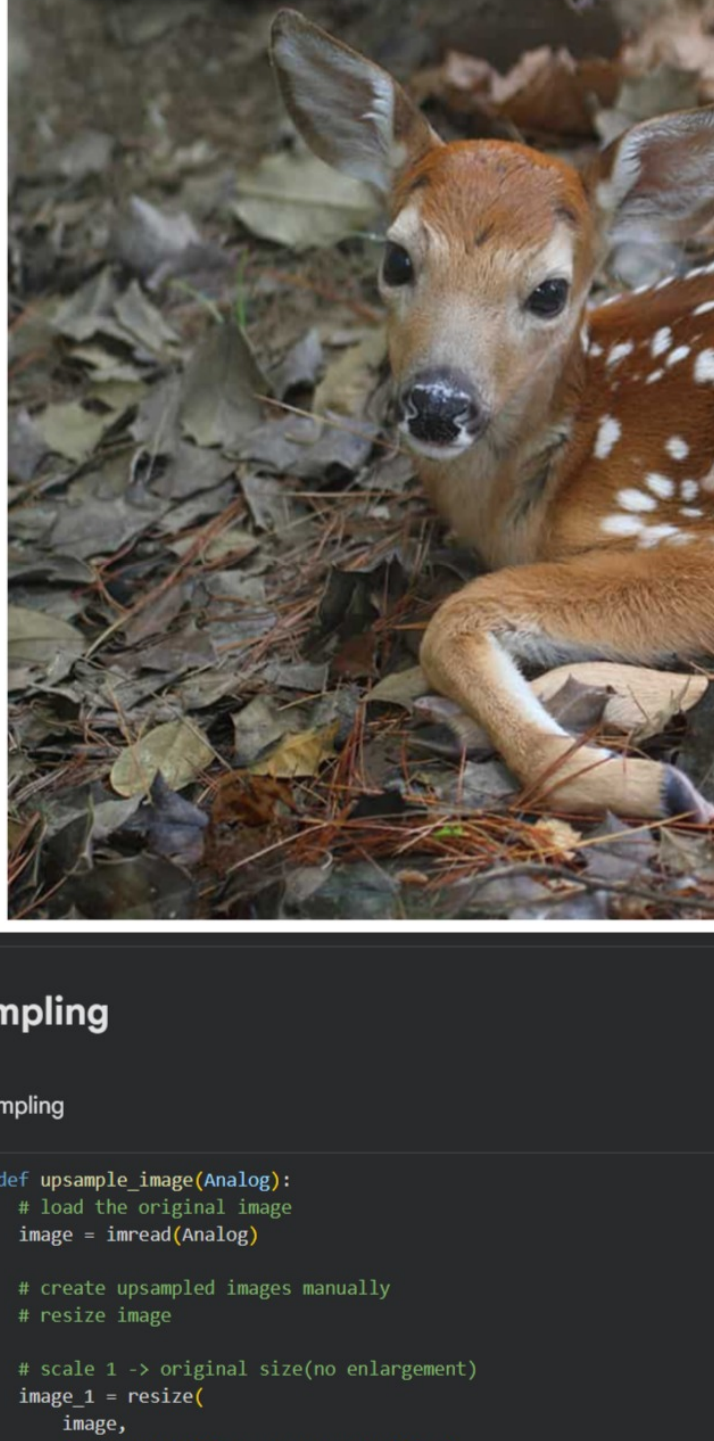
```
import matplotlib.pyplot as plt
from matplotlib.image import imread

def show_analog_image(Analog):
    # load image
    image = imread(Analog)

    # display the image
    plt.figure(figsize=(15,10))
    plt.imshow(image)
    plt.title('Example of an analog image')
    plt.axis('off')
    plt.show()

# display analog image
show_analog_image('Analog.png')
```

Example of an analog image



Digital Image

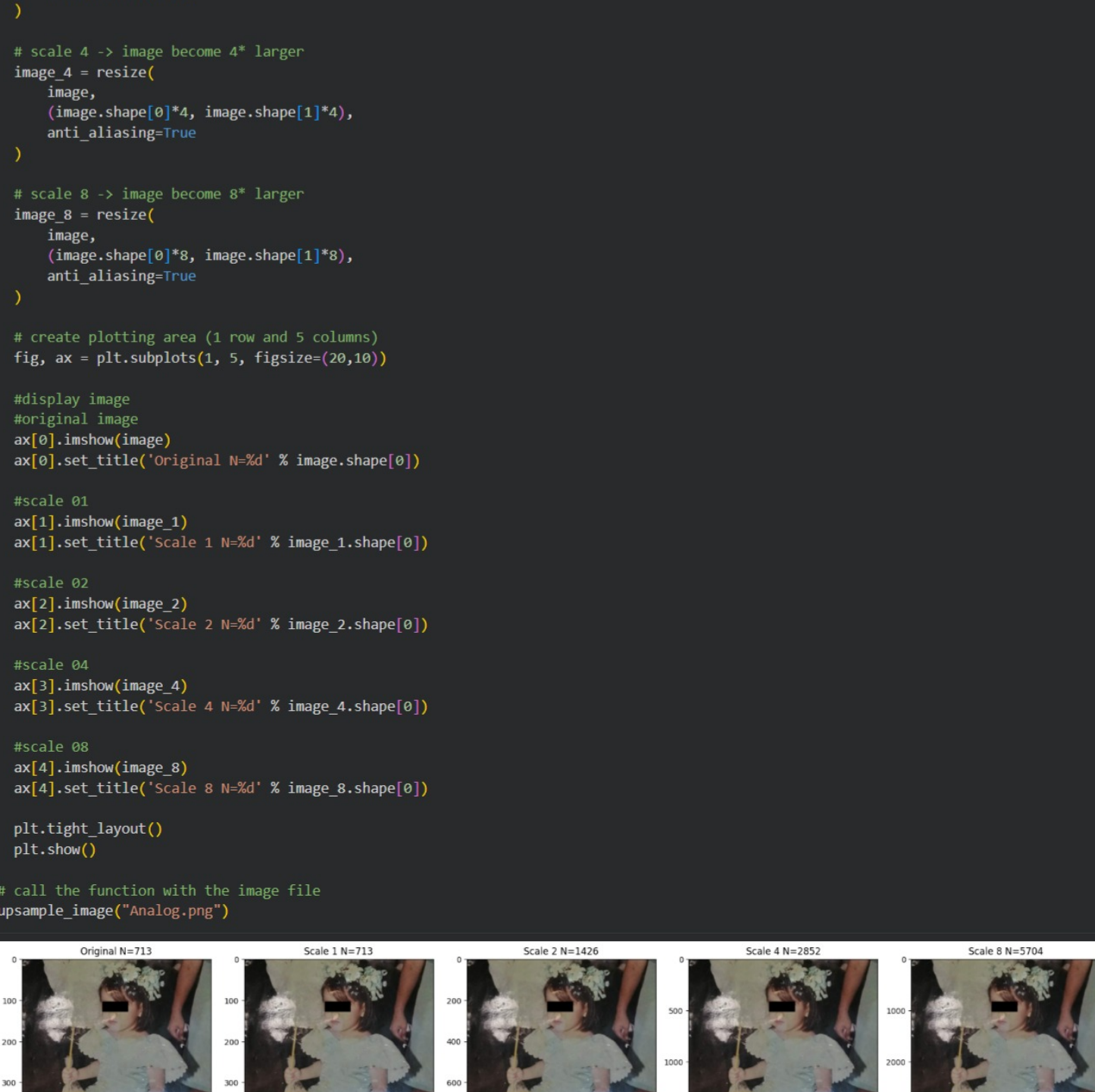
```
import matplotlib.pyplot as plt
from matplotlib.image import imread

def show_analog_image(Digital):
    # load image
    image = imread(Digital)

    # display the image
    plt.figure(figsize=(15,10))
    plt.imshow(image)
    plt.title('Example of a digital image')
    plt.axis('off')
    plt.show()

# display analog image
show_analog_image('Digital.jpg')
```

Example of a digital image



Sampling

Upsampling

```
def upsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create upsampled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = resize(
        image,
        (image.shape[0]*1, image.shape[1]*1),
        anti_aliasing=True
    )

    # scale 2 -> image become 2* larger
    image_2 = resize(
        image,
        (image.shape[0]*2, image.shape[1]*2),
        anti_aliasing=True
    )

    # scale 4 -> image become 4* larger
    image_4 = resize(
        image,
        (image.shape[0]*4, image.shape[1]*4),
        anti_aliasing=True
    )

    # scale 8 -> image become 8* larger
    image_8 = resize(
        image,
        (image.shape[0]*8, image.shape[1]*8),
        anti_aliasing=True
    )

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(20,10))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original N-%d' % image.shape[0])

    #scale 01
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1 N-%d' % image_1.shape[0])

    #scale 02
    ax[2].imshow(image_2)
    ax[2].set_title('Scale 2 N-%d' % image_2.shape[0])

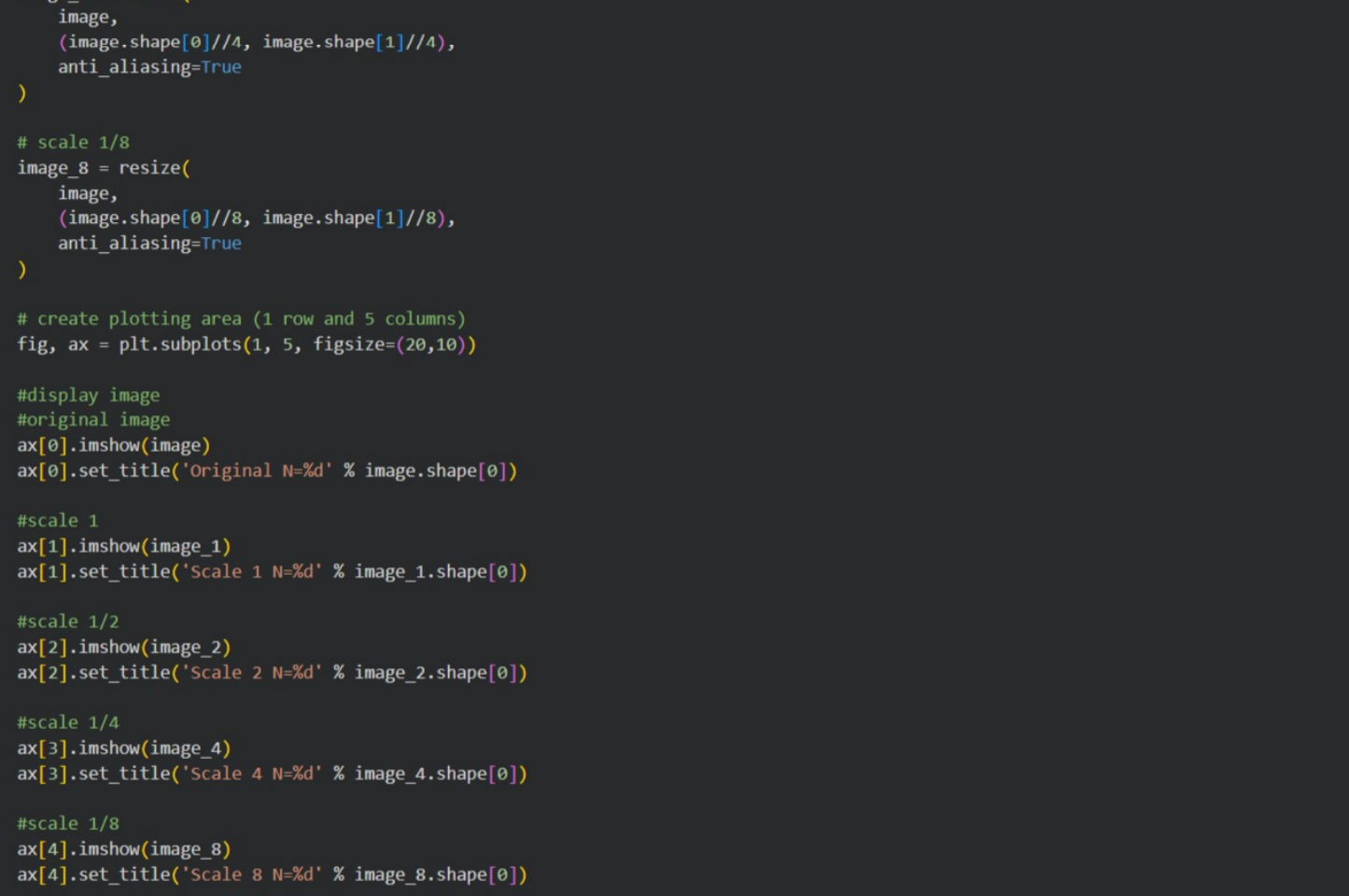
    #scale 04
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4 N-%d' % image_4.shape[0])

    #scale 08
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8 N-%d' % image_8.shape[0])

    plt.tight_layout()
    plt.show()

# call the function with the image file
upsample_image("Analog.png")
```

Original N=713 Scale 1 N=713 Scale 2 N=1426 Scale 4 N=2852 Scale 8 N=5704



Downsampling

```
def downsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create downsampled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = resize(
        image,
        (image.shape[0], image.shape[1]),
        anti_aliasing=True
    )

    # scale 1/2
    image_2 = resize(
        image,
        (image.shape[0]//2, image.shape[1]//2),
        anti_aliasing=True
    )

    # scale 1/4
    image_4 = resize(
        image,
        (image.shape[0]//4, image.shape[1]//4),
        anti_aliasing=True
    )

    # scale 1/8
    image_8 = resize(
        image,
        (image.shape[0]//8, image.shape[1]//8),
        anti_aliasing=True
    )

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(20,10))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original N-%d' % image.shape[0])

    #scale 1
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1 N-%d' % image_1.shape[0])

    #scale 1/2
    ax[2].set_title('Scale 2 N-%d' % image_2.shape[0])
    ax[2].imshow(image_2)

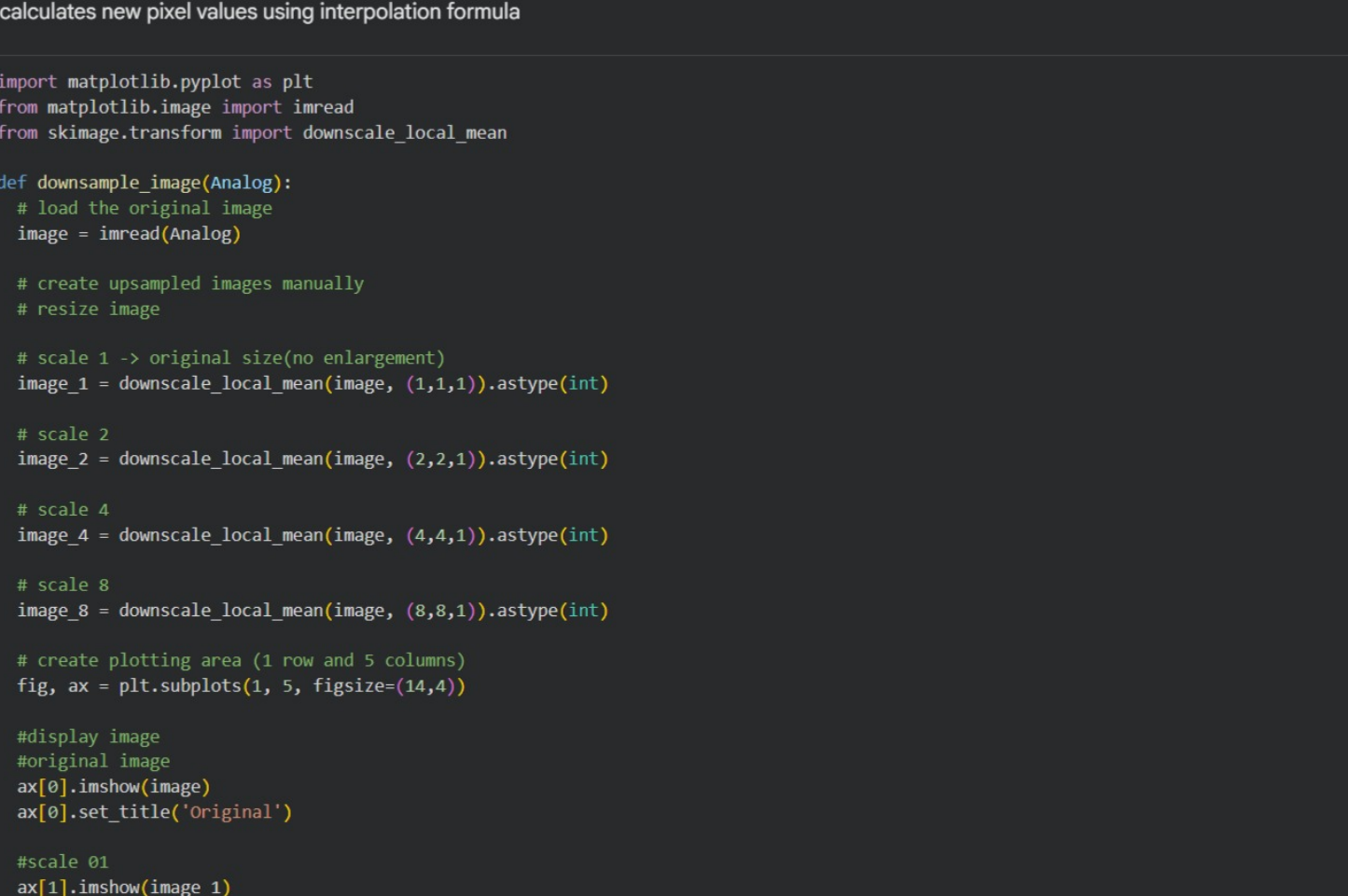
    #scale 1/4
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4 N-%d' % image_4.shape[0])

    #scale 1/8
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8 N-%d' % image_8.shape[0])

    plt.tight_layout()
    plt.show()

# call the function with the image file
downsample_image("Analog.png")
```

Original N=713 Scale 1 N=713 Scale 2 N=356 Scale 4 N=178 Scale 8 N=89



resize() Downsampling

What it does:

- Uses interpolation (bilinear / bicubic)
- Estimates new pixel values from nearby pixels
- Works like geometric rescaling

Effect:

- Image looks smoother
- Edges may blur
- Values are interpolated (not true averaging)

👉 It calculates new pixel values using interpolation formula

```
import matplotlib.pyplot as plt
from matplotlib.image import imread
from skimage.transform import downscale_local_mean

def downsample_image(Analog):
    # load the original image
    image = imread(Analog)

    # create upsampled images manually
    # resize image

    # scale 1 -> original size(no enlargement)
    image_1 = downscale_local_mean(image, (1,1,1)).astype(int)

    # scale 2
    image_2 = downscale_local_mean(image, (2,2,1)).astype(int)

    # scale 4
    image_4 = downscale_local_mean(image, (4,4,1)).astype(int)

    # scale 8
    image_8 = downscale_local_mean(image, (8,8,1)).astype(int)

    # create plotting area (1 row and 5 columns)
    fig, ax = plt.subplots(1, 5, figsize=(14,4))

    #display image
    #original image
    ax[0].imshow(image)
    ax[0].set_title('Original')

    #scale 01
    ax[1].imshow(image_1)
    ax[1].set_title('Scale 1')

    #scale 02
    ax[2].imshow(image_2)
    ax[2].set_title('Scale 2')

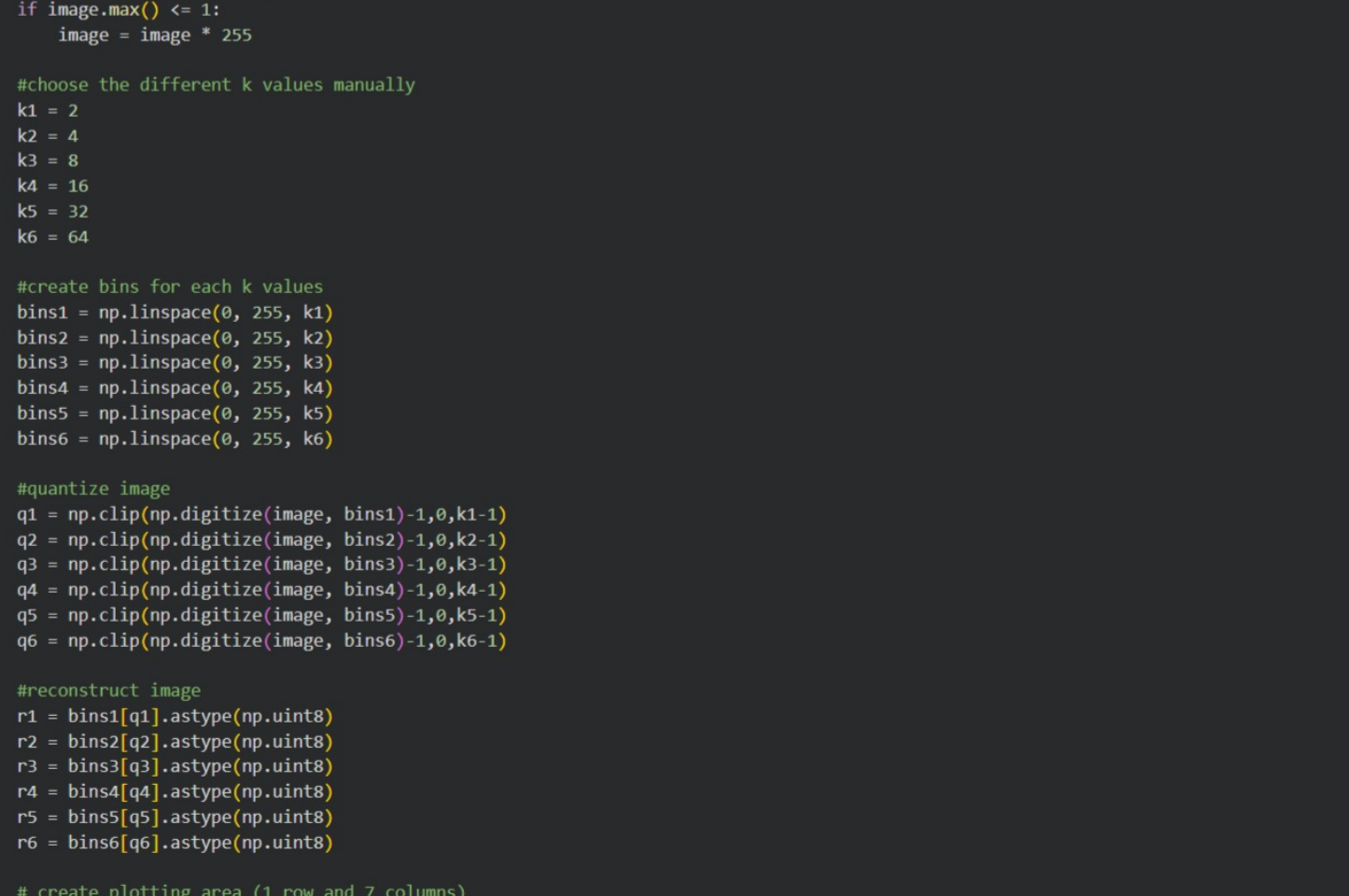
    #scale 04
    ax[3].imshow(image_4)
    ax[3].set_title('Scale 4')

    #scale 08
    ax[4].imshow(image_8)
    ax[4].set_title('Scale 8')

    plt.tight_layout()
    plt.show()

# call the function with the image file
downsample_image("Analog.png")
```

Original Scale 1 Scale 2 Scale 4 Scale 8



👉 downscale_local_mean()

What it does:

- Divides image into blocks (2x2, 4x4, etc.)
- Computes the mean (average) of each block
- Replaces the block with one pixel

Effect:

- Preserves overall brightness
- Reduces aliasing
- Looks slightly blurred but more mathematically correct

👉 It performs low-pass filtering + subsampling

Quantization

```
import matplotlib.pyplot as plt
from matplotlib.image import imread
import numpy as np

def quantize_image(Analog):
    #load the original image
    image = imread(Analog)

    #convert if image range is 0-1
    if image.max() <= 1:
        image = image * 255

    #choose the different k values manually
    k1 = 2
    k2 = 4
    k3 = 8
    k4 = 16
    k5 = 32
    k6 = 64

    #create bins for each k values
    bins1 = np.linspace(0, 255, k1)
    bins2 = np.linspace(0, 255, k2)
    bins3 = np.linspace(0, 255, k3)
    bins4 = np.linspace(0, 255, k4)
    bins5 = np.linspace(0, 255, k5)
    bins6 = np.linspace(0, 255, k6)

    #quantize image
    q1 = np.clip(np.digitize(image, bins1)-1,0,k1-1)
    q2 = np.clip(np.digitize(image, bins2)-1,0,k2-1)
    q3 = np.clip(np.digitize(image, bins3)-1,0,k3-1)
    q4 = np.clip(np.digitize(image, bins4)-1,0,k4-1)
    q5 = np.clip(np.digitize(image, bins5)-1,0,k5-1)
    q6 = np.clip(np.digitize(image, bins6)-1,0,k6-1)

    #reconstruct image
    r1 = bins1[q1].astype(np.uint8)
    r2 = bins2[q2].astype(np.uint8)
    r3 = bins3[q3].astype(np.uint8)
    r4 = bins4[q4].astype(np.uint8)
    r5 = bins5[q5].astype(np.uint8)
    r6 = bins6[q6].astype(np.uint8)

    # create plotting area (1 row and 7 columns)
    fig, ax = plt.subplots(1, 7, figsize=(14,4))

    ax[0].imshow(image.astype(np.uint8))
    ax[0].set_title('Original')

    ax[1].imshow(r1)
    ax[1].set_title('k = 2')

    ax[2].imshow(r2)
    ax[2].set_title('k = 4')

    ax[3].imshow(r3)
    ax[3].set_title('k = 8')

    ax[4].imshow(r4)
    ax[4].set_title('k = 16')

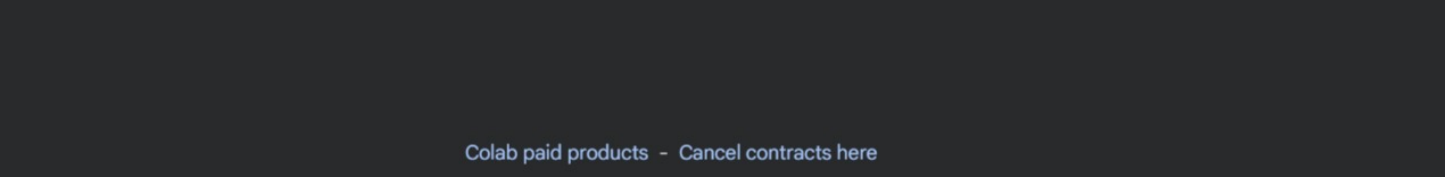
    ax[5].imshow(r5)
    ax[5].set_title('k = 32')

    ax[6].imshow(r6)
    ax[6].set_title('k = 64')

    plt.tight_layout()
    plt.show()

# call the function
quantize_image("Analog.png")
```

Original k = 2 k = 4 k = 8 k = 16 k = 32 k = 64



```
import matplotlib.pyplot as plt
from matplotlib.image import imread

def show_image_color_types(Digital):
    #load the original image
    image = imread(Digital)

    #convert to grayscale
    gray = rgb2gray(image)

    #convert grayscale to binary
    binary = img_as_uint(gray > 0.5)

    #convert grayscale to color (3 channel)
    color = gray2rgb(gray)

    #Apply jet colormap
    jet = plt.cm.jet(gray)

    #display images
    fig, ax = plt.subplots(1,5, figsize=(15,5))

    ax[0].imshow(image)
    ax[0].set_title("Original")

    ax[1].imshow(gray, cmap='gray')
    ax[1].set_title("grayscale")

    ax[2].imshow(binary, cmap='gray')
    ax[2].set_title("binary")

    ax[3].imshow(color)
    ax[3].set_title("gray to color")

    ax[4].imshow(jet)
    ax[4].set_title("jet colormap")

    for a in ax:
        a.axis("off")

    plt.tight_layout()
    plt.show()

# call the function
show_image_color_types("Digital.jpg")
```

Original Grayscale Binary Gray to Color jet Colormap

Colab paid products - Cancel contracts here